

Expense Tracker

Full-Stack Web Application | React.js + Node.js + Express.js + MySQL

Project Overview

Expense Tracker is a full-stack web application created to demonstrate how a modern frontend, backend, REST API, authentication system, and relational database work together. A user can create an account, log in securely, add daily expenses, view all personal expenses, see the total amount spent, delete expenses, and log out. The project was designed as both a portfolio project and a practical learning project, so its structure follows a clear separation between the React frontend and the Node.js/Express backend.

The frontend is built with React.js and Vite. React manages the user interface, form data, dashboard state, and communication with the backend through the Fetch API. The backend is built with Node.js and Express.js and exposes REST endpoints for signup, login, adding expenses, retrieving expenses, calculating totals, and deleting expenses. MySQL stores users and expenses permanently. Passwords are hashed with bcrypt, while JSON Web Tokens (JWT) are used to authenticate requests to protected expense routes.

Main Application Flow



When the user performs an action in the browser, React sends an HTTP request to the REST API. Express receives the request, validates it, runs the appropriate controller logic, and communicates with MySQL. The backend then sends a JSON response to React, and React updates the screen. This request-response cycle is the central idea behind the project.

Core Features

Authentication: signup, login, bcrypt password hashing, JWT generation and verification, protected API routes, expired/invalid token handling, and logout.

Expense management: add an expense with a name, amount, and description; retrieve the logged-in user's expenses; delete an expense; and calculate the user's total spending.

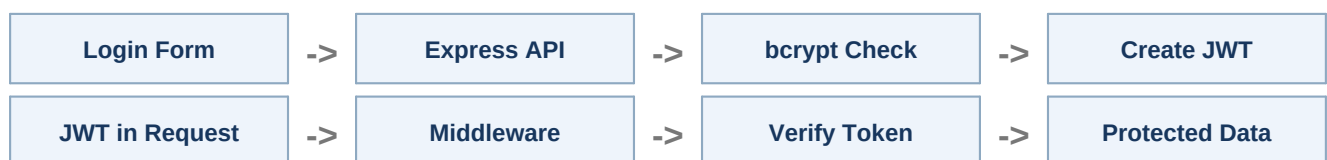
Validation and security: frontend and backend input validation, parameterized MySQL queries, environment variables for secrets, and user-specific database queries based on the verified JWT.

Technology & Architecture

The application uses a common full-stack architecture. React is responsible for what the user sees and interacts with. Node.js provides the JavaScript runtime for the server, while Express.js provides routing and middleware for the REST API. MySQL stores relational data. The **mysql2** package connects Express to MySQL. **bcrypt** protects stored passwords, **jsonwebtoken** creates and verifies JWTs, **dotenv** loads private environment variables, and **CORS** allows the frontend and backend development servers to communicate.

Technology	Purpose in this project
React.js + Vite	Frontend interface, forms, state and dashboard
Node.js + Express.js	Backend server and REST API
MySQL + mysql2	Persistent users and expense data
bcrypt	Password hashing
JWT	Authentication for protected routes
Fetch API	Frontend-to-backend HTTP requests

Authentication Flow



During signup, the backend validates the user's details and hashes the password before saving it. During login, the submitted password is compared with the stored hash. If it matches, the server creates a JWT containing the user's identity. For protected expense requests, the frontend sends the token in the Authorization header. Authentication middleware verifies the token and places the decoded user information in **req.user**. The expense controller then uses **req.user.id** instead of trusting a user ID supplied by the browser.

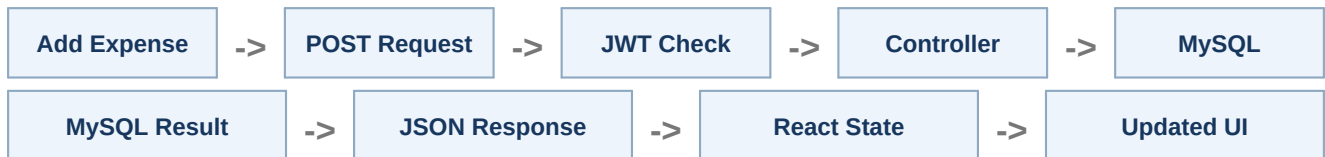
User Data Isolation

Every expense is linked to a user through **user_id**. Expense queries include the authenticated user's ID, so User A receives User A's expenses and User B receives User B's expenses. The delete operation also checks both the expense ID and user ID. This is an important authorization rule because knowing another expense ID should not allow a user to delete someone else's data.

Project Structure, Setup & Learning Value

The repository is divided into **front-end** and **back-end** folders. The React side contains the Login, Signup, Dashboard and application components together with their CSS. The backend separates configuration, routes, controllers and authentication middleware. This separation makes the code easier to understand, debug and extend. The database contains a users table and an expenses table connected by a foreign-key relationship.

Expense Request Flow



For example, when a user adds an expense, React first validates the form and sends the expense data with the JWT to the backend. The authentication middleware verifies the token, the controller validates the amount and other values, and MySQL inserts the expense using the authenticated user's ID. After a successful response, React reloads the expense list and total, so the dashboard immediately reflects the new data.

Running the Project

To run the project locally, install Node.js, npm and MySQL. Create the **expense_tracker** database and its users and expenses tables. In the backend, run **npm install**, create a private **.env** file containing the database settings and JWT secret, and start the server. In the frontend, run **npm install** followed by **npm run dev**. The real **.env** file and **node_modules** folders must not be committed to GitHub; the repository should contain a safe **.env.example** instead.

What This Project Demonstrates

This project demonstrates the complete path from a browser interface to a database: React components and state, forms and event handling, asynchronous Fetch requests, REST endpoints, Express routes, controllers, middleware, SQL queries, relational data, authentication, authorization, validation and error handling. These concepts are reusable. Once the architecture is understood, the same pattern can be adapted to task managers, notes applications, inventory systems, employee management systems, booking applications and other CRUD-based web applications.

Future improvements: edit expenses, categories, expense dates, search and filters, monthly analytics, charts, pagination, React Router, Swagger/OpenAPI documentation, automated tests and production deployment.

Purpose: A compact GitHub project overview for recruiters, interviewers, learners and developers who want to quickly understand what the application does, how the major technologies connect, and what full-stack concepts are demonstrated.